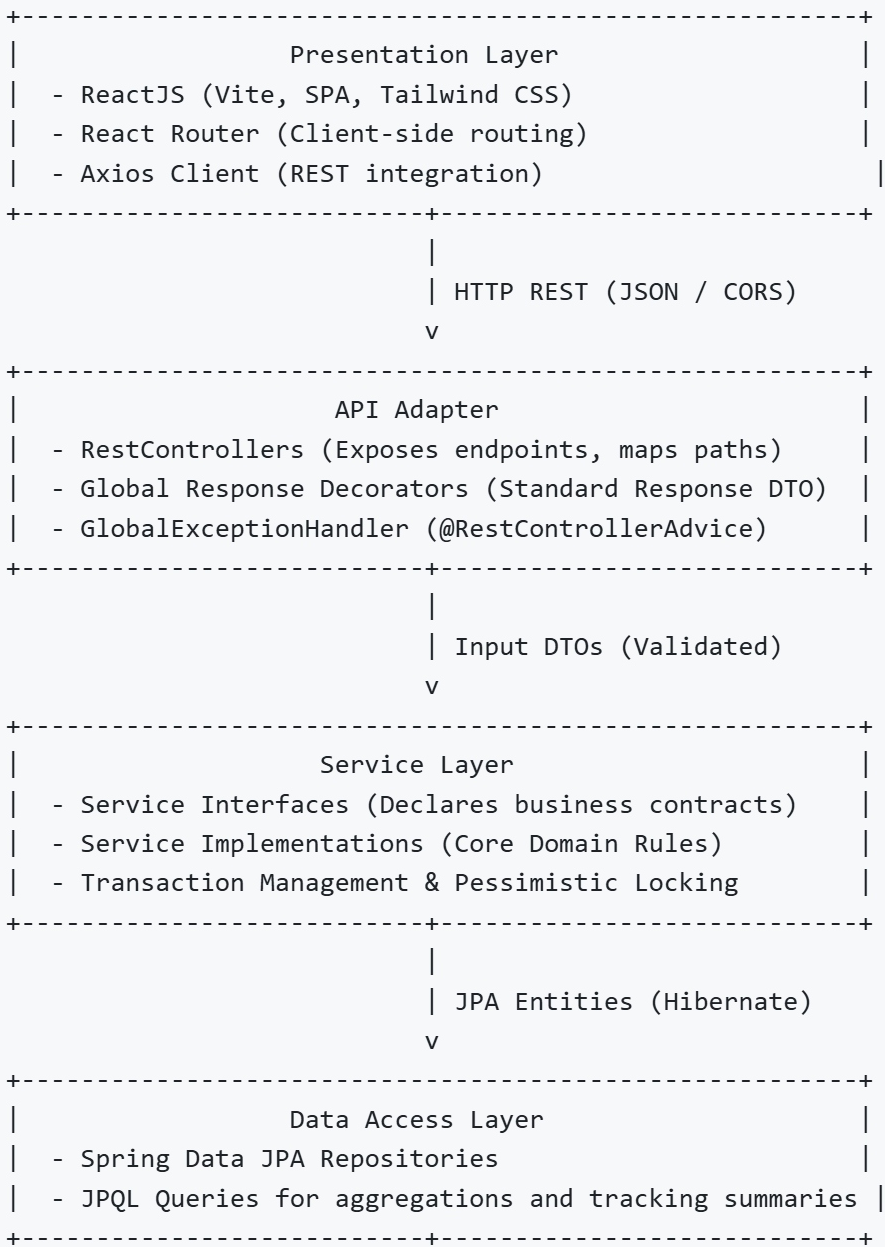


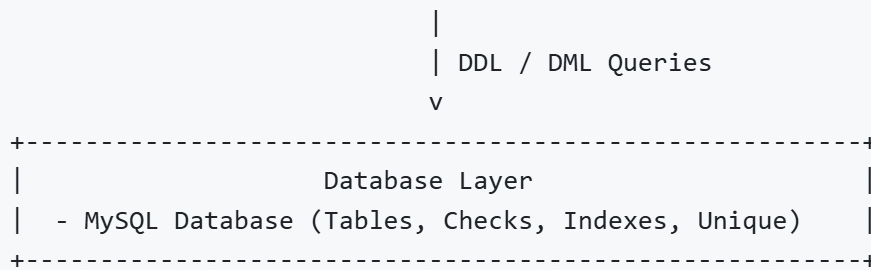
Department Expense Approval System - Software Architecture & Design Specification

This document outlines the complete architectural design, database schematics, API structures, business workflows, and decision rationales for the **Department Expense Approval System**.

1. Complete Software Architecture

The application adopts a **Layered Architecture** adhering to **Clean Architecture** and **SOLID Principles**. This structure isolates the core business rules from external frameworks, database operations, and user interface delivery systems.





Key Architectural Pillars:

- Unidirectional Flow & DTO Protection:** Entities never escape the Service boundary. Controllers receive Request DTOs and return Response DTOs. A dedicated Mapping layer translates between Entities and DTOs.
- Pessimistic Concurrency Model:** Concurrency-safe budget verification uses pessimistic write locking (`FOR UPDATE`) on the target department's monthly budget to prevent over-allocation of funds during rapid concurrent reviews.
- Enterprise Auditing:** Integrates JPA Auditing listeners (`@CreatedDate` , `@LastModifiedDate`) to manage operational logs (`createdAt` , `updatedAt` , `reviewedAt`) transparently.
- Unified API Contract:** All controller outputs are wrapped inside a generic `ApiResponse<T>` envelope containing status indicators, timestamp, custom messages, and optional validation/pagination metadata.

2. Folder Structure

Backend Package Layout (Maven / Spring Boot 3.x)

```
com.company.expenseapproval
├── config
│   ├── OpenApiConfig.java           # Swagger / OpenAPI documentation configurations
│   ├── WebConfig.java               # CORS policies, interceptors, and MVC rules
│   └── JpaAuditingConfig.java        # Enables Spring JPA Auditing listeners
├── controller
│   ├── BudgetController.java         # Endpoints: /api/budgets
│   └── ExpenseClaimController.java   # Endpoints: /api/expense-claims
├── dto
│   ├── request
│   │   ├── BudgetRequestDTO.java     # Creation & validation payload for budgets
│   │   ├── ClaimReviewRequestDTO.java # Approval/Rejection payload
│   │   └── ExpenseClaimRequestDTO.java # Submission payload for claims
│   └── response
│       ├── ApiResponse.java          # Envelope for unified API responses
│       └── BudgetResponseDTO.java     # Serialized budget output
```

```

├── ExpenseClaimResponseDTO.java # Serialized expense claim output
├── FinanceSummaryDTO.java      # Financial metrics response
├── PaginatedResponse.java      # Custom wrapper for Spring Page collections
├── entity
│   ├── BaseAuditEntity.java    # Abstract class holding @CreatedDate & @LastModified
│   ├── DepartmentBudget.java   # JPA Entity mapped to department_budgets
│   └── ExpenseClaim.java       # JPA Entity mapped to expense_claims
├── enums
│   ├── Department.java         # IT, HR, FINANCE, SALES, MARKETING
│   ├── ExpenseCategory.java    # TRAVEL, FOOD, MEDICAL, OFFICE_SUPPLIES, TRAINING
│   └── ExpenseStatus.java      # PENDING, APPROVED, REJECTED
├── exception
│   ├── BudgetExceededException.java # Exceeding remaining budget on approval
│   ├── DuplicateBudgetException.java # Month/Year budget already defined for department
│   ├── ErrorResponse.java        # Standard payload structure for errors
│   ├── GlobalExceptionHandler.java # Centralized @RestControllerAdvice
│   ├── InvalidExpenseStateException.java # Attempt to modify historical claims
│   └── ResourceNotFoundException.java # Entity target missing
├── mapper
│   ├── BudgetMapper.java        # Hand-crafted or MapStruct mapper definitions
│   └── ExpenseClaimMapper.java   # Mappings between Claim entity and DTOs
├── repository
│   ├── DepartmentBudgetRepository.java # JPA repo supporting locking methods
│   └── ExpenseClaimRepository.java     # JPA repo supporting page filters and aggregation
├── service
│   ├── DepartmentBudgetService.java    # Business definitions for budgets
│   ├── ExpenseClaimService.java        # Business definitions for claims and flow decisions
│   └── impl
│       ├── DepartmentBudgetServiceImpl.java # Logic implementations
│       └── ExpenseClaimServiceImpl.java     # Lock handling & budget evaluation
└── util
    └── LogMessages.java                # Reusable logging trace text variables

```

Frontend Folder Structure (ReactJS + Vite)

```

src/
├── assets/
│   ├── logo.svg
│   └── images/
├── components/
│   ├── common/
│   │   ├── Button.jsx          # Custom Tailwind styled button components
│   │   └── Card.jsx            # Content wrapper

```

├── Card.jsx	# Card component wrapper
├── FormInput.jsx	# Form fields with validation messaging
├── FormSelect.jsx	# Custom styled select boxes
├── Modal.jsx	# Confirmations (e.g. Approve/Reject alerts)
├── Navbar.jsx	# Header with client role toggle hook
├── Pagination.jsx	# Page navigators
└── Toast.jsx	# Toast containers for notification alerts
├── budget/	
│ ├── BudgetForm.jsx	# Manage monthly budget allocation
│ └── BudgetList.jsx	# Table displaying department budgets
├── claims/	
│ ├── ClaimCard.jsx	# Card layout showing claim metrics
│ ├── ClaimFilter.jsx	# Combined search, status, and category selectors
│ ├── ClaimForm.jsx	# React Hook Form for claim creation
│ └── ClaimList.jsx	# Claims list with paginated support
└── summary/	
│ ├── SummaryMetricCard.jsx	# Displays budget usage percentages
│ └── SummaryTable.jsx	# Monthly breakdown sheet view
├── context/	
│ └── RoleContext.jsx	# Global state simulation for current user role
├── hooks/	
│ ├── useFetch.js	# Dynamic data fetching hook
│ └── useDebounce.js	# Delays input searches to avoid API flooding
├── pages/	
│ ├── BudgetPage.jsx	# Budget dashboard
│ ├── ClaimReviewPage.jsx	# Review flow layout for managers
│ ├── ClaimSubmissionPage.jsx	# Creation form layout for employees
│ ├── DashboardPage.jsx	# Entry screen displaying statistics
│ └── SummaryPage.jsx	# Metric table page
├── services/	
│ ├── api.js	# Base Axios configuration
│ ├── budgetService.js	# Axios endpoint actions for budgets
│ └── expenseService.js	# Axios endpoint actions for claims
├── styles/	
│ └── index.css	# Global CSS & Tailwind directives
├── utils/	
│ ├── constants.js	# Lists categories, static departments, and routes
│ └── formatters.js	# Date formatters and currency parsers
├── App.jsx	# Context wrapper & router setup
├── main.jsx	# React bootstrap entry point
└── routes.jsx	# Router layouts

3. Database Design

The schema is built for a MySQL database engine, utilizing InnoDB for row-level locks, transactional integrity, and referential constraints.

```
-- Create Department Budgets Table
CREATE TABLE department_budgets (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    department VARCHAR(50) NOT NULL,
    budget_month INT NOT NULL,
    budget_year INT NOT NULL,
    budget_amount DECIMAL(15, 2) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    -- Table Constraints
    CONSTRAINT chk_budget_month CHECK (budget_month BETWEEN 1 AND 12),
    CONSTRAINT chk_budget_year CHECK (budget_year >= 2000),
    CONSTRAINT chk_budget_amount CHECK (budget_amount > 0.00),
    CONSTRAINT uq_dept_month_year UNIQUE (department, budget_month, budget_year)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Create Expense Claims Table
CREATE TABLE expense_claims (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    employee_name VARCHAR(100) NOT NULL,
    department VARCHAR(50) NOT NULL,
    category VARCHAR(50) NOT NULL,
    amount DECIMAL(15, 2) NOT NULL,
    expense_date DATE NOT NULL,
    description VARCHAR(500) NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    review_remark VARCHAR(255) NULL,
    reviewed_at TIMESTAMP NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    -- Table Constraints
    CONSTRAINT chk_expense_amount CHECK (amount > 0.00),
    CONSTRAINT chk_expense_status CHECK (status IN ('PENDING', 'APPROVED', 'REJECTED')),

    -- Indexes for optimization
    INDEX idx_expense_filtering (department, status, category, expense_date),
    INDEX idx_expense_employee (employee_name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4. Entity-Relationship Diagram (Text Based)

4. Entity Relationship Diagram (Text-Based)



5. API Design

Base URL: `/api`

Endpoint	Method	Payload	Responses	Description
Budgets				
<code>/v1/budgets</code>	POST	BudgetRequestDTO	201 Created , 400 Bad Request , 409 Conflict	Create a new department monthly budget
<code>/v1/budgets</code>	GET	None	200 OK	Fetch all department budgets

Expense Claims				
/v1/expense-claims	POST	ExpenseClaimRequestDTO	201 Created , 400 Bad Request	Submit a claim (Status forced to PENDING)
/v1/expense-claims	GET	Query Params (Filters)	200 OK	Fetch all claims using dynamic filters and pagination
/v1/expense-claims/{id}	GET	None	200 OK , 404 Not Found	Retrieve detail metrics of a claim
/v1/expense-claims/{id}/review	PUT	ClaimReviewRequestDTO	200 OK , 400 Bad Request , 404 Not Found	Approve or Reject a pending expense claim
Finance Summary				
/v1/finance-summary	GET	Query Params (Month & Year)	200 OK , 400 Bad Request	Get monthly summary sheet per department

Dynamic Filtering Parameters (GET /v1/expense-claims)

- department (Enum value matching IT , HR , etc.)
- month (Int value 1-12)
- year (Int value >= 2000)
- category (Enum value matching TRAVEL , FOOD , etc.)
- status (Enum value matching PENDING , APPROVED , REJECTED)
- employeeName (String, partial search match supported)
- page (Int, default: 0)
- size (Int, default: 10)
- sort (String, format: field,direction , default: createdAt,desc)

Custom Error Responses (400 , 404 , 409)

All errors yield a standardized layout structure:

```
{
  "success": false,
  "message": "Resource could not be loaded",
  "errorCode": "RESOURCE_NOT_FOUND",
  "timestamp": "2026-06-30T16:41:22",
  "details": ["ExpenseClaim with ID 42 not found"]
}
```

6. Validation Strategy

I. Request DTO Validations (JSR-380 annotations in Spring Boot)

1. ExpenseClaimRequestDTO:

- `employeeName` : `@NotBlank` , `@Size(min = 2, max = 100)` , `@Pattern(regexp = "^(?!\\s+$).*" , message = "Must not be empty spaces")`
- `department` : `@NotNull(message = "Department is required")` (Validated against Java `Department` Enum)
- `category` : `@NotNull(message = "Category is required")` (Validated against Java `ExpenseCategory` Enum)
- `amount` : `@NotNull` , `@DecimalMin(value = "0.01", message = "Expense amount must be greater than zero")` , `@DecimalMax(value = "100000.00", message = "Expense claim exceeds limit")`
- `expenseDate` : `@NotNull` , `@PastOrPresent(message = "Expense date cannot be in the future")`
- `description` : `@Size(max = 500)`

2. BudgetRequestDTO:

- `department` : `@NotNull`
- `budgetMonth` : `@NotNull` , `@Min(1)` , `@Max(12)`
- `budgetYear` : `@NotNull` , `@Min(2000)`
- `amount` : `@NotNull` , `@DecimalMin(value = "0.01", message = "Budget amount must be greater than zero")`

3. ClaimReviewRequestDTO:

- `status` : `@NotNull` (Value must resolve strictly to `APPROVED` or `REJECTED`)
- `remark` : `@Size(max = 255)` (Business validation logic: mandatory if status is `REJECTED`)

II. Database Level Validations

- Column Nullability: Set `NOT NULL` on essential transaction columns to maintain data completeness.

- Constraints: Database constraint checks (`chk_budget_month` , `chk_budget_amount`) verify invariants at insertion.
- Unique Constraints: `uq_dept_month_year` enforces single allocations per month/year per department at the engine index level.

III. Frontend validations (React Hook Form)

- Direct field validation dynamically provides user UI markers (red highlight borders, disable submits) to reduce network requests.
- Checks field ranges (e.g. date inputs set `max` to local current system date).

7. Business Flow

Concurrency-Safe Budget Validation during Approval

Finance Manager	Server Thread	Database
	 --- PUT /claims/{id}/review --->	
	(status=APPROVED)	
	 -- Begin Transaction ----->	
	 -- SELECT budget FOR UPDATE->	
	(Pessimistic Write Lock)	
	 <-- Return Budget Record ----	
	(Other threads block	
	on this department budget)	
	 -- SUM Approved Claims ----->	
	(Same dept, month, year)	
	 <-- Return Total Approved ---	
	 -- Check:	
	(Sum + Claim) <= Budget	
	 ----[IF EXCEEDED]	
	 ----> Throw BudgetExceeded	
	 ----> Rollback Transaction	
	 ----[IF SUFFICIENT]	
	 ----> Update Claim Status	
	 ----> Save operational logs	
	 ----> Commit Transaction --->	

Core Business Rules:

1. Pristine Claim Invariant:

- `newClaim.status` is forced to `PENDING` by system defaults.

2. State Machine Finality:

- Transitions can only originate from `PENDING`.
- Once status evaluates to `APPROVED` or `REJECTED`, all future transitions are blocked (`InvalidExpenseStateException`).

3. Budget Consumption Invariant:

- Remaining Budget Calculation: `BudgetAmount - SUM(ApprovedClaims)`.
- `PENDING` and `REJECTED` claims are excluded.

8. Project Workflow

[PHASE 1: Contract & Init]

- Verify environments (Java 17, Maven, MySQL 8.x)
- Generate database instance and user accounts
- Design Swagger/OpenAPI documentation specifications

[PHASE 2: Database Migration]

- Build schema configuration scripts
- Establish entity beans with Hibernate auditing support

[PHASE 3: Service & Core Logic]

- Create services and repositories
- Construct concurrency locks and write JUnit transaction scenarios

[PHASE 4: Web layer & Validation]

- Configure REST controller bindings
- Set up validation handlers and Response envelope wrapper mapping filters

[PHASE 5: Frontend Layouts]

- Initialize Vite React project
- Scaffold pages with Tailwind styling
- Build state mechanisms for client roles

[PHASE 6: Axios integrations & E2E Validation]

- Bind client with backend services
- Validate validations and mock concurrent updates

9. Development Roadmap

- **Milestone 1: Backend Architecture Setup** (Duration: Day 1)
 - Framework skeleton generation, package organization, Maven POM dependencies setup.
 - Audit entity configuration and local MySQL connection setup.
- **Milestone 2: Entities & Core Service Logic** (Duration: Day 2)
 - Write domain beans, JSR validation mappings, database tables script creation.
 - Implement Service structures. Write `FOR UPDATE` transaction lock controls.
- **Milestone 3: Presentation REST Layer & Exception Advising** (Duration: Day 3)
 - Implement controllers, DTO specifications, custom exceptions.
 - Integrate OpenAPI documentation. Mock validations using Swagger/Postman profiles.
- **Milestone 4: React UI Client Core** (Duration: Day 4)
 - Scaffold SPA, set up Tailwind styles, React Router definitions, and custom Toast notification overlays.
 - Create component modules (forms, grids, context roles toggle switches).
- **Milestone 5: Hookups & Final Verification** (Duration: Day 5)
 - Connect Axios HTTP request clients.
 - Write manual validation integration verification steps: assert concurrency locks, budget overruns, and input field boundaries.

10. Explanation of Every Design Decision

1. Why map Enums as String (`EnumType.STRING`) in JPA?

- *Decision:* Persist enums using their text representations (e.g. `IT` , `APPROVED`) instead of integers (`0` , `1`).
- *Rationale:* Storing ordinal integers is brittle; if we insert or rearrange the enums, old database values map incorrectly. Strings remain human-readable directly in MySQL query tools and ensure future compatibility.

2. Why Pessimistic Locking (`PESSIMISTIC_WRITE`) over Optimistic?

- *Decision:* Use database-level row locks during claims review instead of version variables.
- *Rationale:* In finance operations, double-approving claims that share a budget is a critical risk. Optimistic locking fails *after* computations are done when saving the transaction, forcing the user to retry. Pessimistic locks serialize approval threads for the specific department budget, avoiding race conditions and ensuring that budget balances remain accurate during processing.

3. Why decouple Entity relationships and not use hard Foreign Keys on Enums?

- *Decision:* Enums are mapped as column values on both tables without direct table references.
- *Rationale:* Relational foreign keys between a static configuration list (represented as Java Enums) and operational records are unnecessary overhead. It decouples the tables

Java Entities, and operational records are unnecessary overhead. It decouples the tables database-wise, allowing the application layer to manage classification domains while saving index space.

4. Why use React Hook Form over native state binding?

- *Decision:* Form bindings are handled using React Hook Form library.
- *Rationale:* Expense submissions use multiple input fields. Controlled inputs using native React state trigger component re-renders on every keystroke. React Hook Form uses uncontrolled inputs under the hood, dramatically increasing performance and simplifying validation configurations.

5. Why implement a Global API Response Envelope?

- *Decision:* Wrap all controllers outputs within `ApiResponse<T>`.
- *Rationale:* Consuming disparate payloads on the frontend leads to fragile client parsing logic. Providing a structured signature containing `success`, `message`, `data`, and `timestamp` guarantees that the client application can handle errors and paginated content uniformly.

6. Why enforce validation on both Frontend and Backend layers?

- *Decision:* Replicate validation constraints in React components and Spring DTO objects.
- *Rationale:* Frontend validation provides instant feedback and saves network calls. Backend validation is the ultimate source of truth, securing the system against direct, malformed REST requests (e.g. via Postman or terminal scripts).

7. Why use DTOs instead of passing JPA Entities to the client?

- *Decision:* Convert Entities to DTOs in service boundary layers.
- *Rationale:* Directly exposing JPA entities invites performance leaks (like lazy-loading trigger bugs) and exposes internal database columns. It also prevents the client from accidentally sending modified values for auditing fields (like `createdAt`).